

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Report

Laboratory work No. 3

The Command Line Shell, Pipes and Text Filters

Ciur Alexandrina FAF-241

Checked by:
Reitamn Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chișinău – 2026

Contents

1 ALGORITHM ANALYSIS **1**
- 1.1 Objective . . . 1
- 1.2 Tasks . . . 1
- 1.3 Theoretical Notes . . . 1
- 1.4 Introduction . . . 2
- 1.5 Comparison Metric . . . 2
- 1.6 Input Format . . . 2

2 IMPLEMENTATION **3**
- 2.1 Part 1: Move Fast, Find Help . . . 3
  - 2.1.1 Tab Completion . . . 3
  - 2.1.2 History Search . . . 3
  - 2.1.3 Built-in or External . . . 3
  - 2.1.4 Finding Help . . . 3
  - 2.1.5 Observations . . . 4
- 2.2 Part 2: What the Shell Does to the Line . . . 4
  - 2.2.1 Predictions and Real Outputs . . . 4
  - 2.2.2 Observations . . . 6
- 2.3 Part 3: Redirection, Chaining and Jobs . . . 6
  - 2.3.1 Observations . . . 7
- 2.4 Part 4: Answering Questions with One-Liners . . . 8
  - 2.4.1 Q1: Number of Failed Requests . . . 8
  - 2.4.2 Q2: Different Requested Pages . . . 8
  - 2.4.3 Q3: Number of Requests for Each Page . . . 8
  - 2.4.4 Q4: Users with the Most Failed Requests . . . 8
  - 2.4.5 Q5: Requests Made by user3 . . . 9
  - 2.4.6 Q6: Last Three Failed Requests . . . 9
  - 2.4.7 Q7: Login Shells from /etc/passwd . . . 9
  - 2.4.8 Q8: Comparing the Number of Lines . . . 9
- 2.5 Part 5: The Bash Report Script . . . 10
  - 2.5.1 Explanation of the Script . . . 10
  - 2.5.2 Running the Script . . . 10
  - 2.5.3 Observations . . . 11

3 CONCLUSIONS **12**

1 ALGORITHM ANALYSIS

1.1 Objective

The objective of this laboratory work is to practice using the Linux command line shell: moving fast in bash, predicting what the shell does to a command line before running it (globbing, quoting, expansions), redirecting and chaining commands, and answering questions about data with pipelines of small text filters.

1.2 Tasks

1. Use Tab completion and history search; identify which commands are shell built-ins and which are external programs; inspect `PATH`; find help with `man`, `-help` and `help`.
2. Predict, run and explain how the shell expands a command line: wildcards, quoting, variables, brace, arithmetic and command substitution.
3. Practice redirection (`>`, `»`, `<`, `2>`, `2>&1`), chaining operators (`;`, `&&`, `||`) and background jobs.
4. Use pipelines to process `access.log` and `/etc/passwd` and answer questions Q1–Q8 with one-liners.
5. Create a Bash script `report.sh` that receives a log file as an argument and prints the total number of requests and the number of failed requests for every user.

1.3 Theoretical Notes

A shell is a command interpreter that allows a user to communicate with the operating system. It reads a command line, transforms it (expansions, quoting, redirections), and only then executes the program with the resulting arguments.

A pipe is represented by the character `|`. It connects the standard output of one command to the standard input of another command.

For example:

```
grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c
```

The commands in this pipeline perform the following operations:

- `grep FAIL access.log` selects lines containing `FAIL`;
- `cut -d' ' -f3` extracts the third field;
- `sort` sorts the extracted values;
- `uniq -c` counts equal consecutive values.

Every process has three standard streams: standard input (0), standard output (1) and standard error (2). Redirections change where these streams point, and every command returns an exit code (0 for success, non-zero for failure), which is used by `&&`, `||` and `$?`.

The most important commands used in this laboratory are:

- `grep` – searches for lines matching a pattern;

- `cut` – extracts fields or characters from each line;
- `sort` – sorts lines of text;
- `uniq` – removes or counts repeated lines;
- `wc` – counts lines, words, or characters;
- `head`, `tail` – display the first / last lines;
- `tr` – translates characters;
- `ls` – lists directory contents.

1.4 Introduction

The input file used in Part 4 is an artificial access log named `access.log` with 500 lines. Each line has the format `date time user result page`, for example:

```
2026-10-05 10:01 user1 OK /page1
```

The laboratory also uses the file `/etc/passwd` to demonstrate how system account information can be processed using text filters. The work was done on Ubuntu in a VirtualBox virtual machine, user `vboxuser`, in the directory `~/os-lab3`.

1.5 Comparison Metric

For Parts 1–3, a prediction was written before running each command and then compared with the real output. For Parts 4–5, the correctness is verified by cross-checking the results:

- the request counts per page (Q3) must add up to 500;
- the failed requests per user (Q4) must add up to the total number of failed requests (Q1);
- the output of `report.sh` must match Q1 and Q4.

1.6 Input Format

The script receives the log file as an input argument:

```
./report.sh access.log
```

The log file is a text file with one request per line and fields separated by spaces. The script expects the users `user0` to `user6` and verifies that the file exists before processing it.

2 IMPLEMENTATION

2.1 Part 1: Move Fast, Find Help

2.1.1 Tab Completion

Typing `ls /usr/sh` and pressing Tab completed the path to `/usr/share/`, because only one entry in `/usr` starts with `sh`. Pressing Tab a second time listed all entries inside the directory; since there were many, bash first asked `Display all 252 possibilities? (y or n)`.

Typing `cd /et` and pressing Tab once completed directly to `cd /etc/`, because there is exactly one match.

Conclusion: one Tab completes when there is a single match; two Tabs list all candidates when there are several.

2.1.2 History Search

After running `pwd`, `date` and `whoami`, pressing Ctrl+R and typing `pw` displayed:

```
(reverse-i-search)`pw': pwd
```

Ctrl+R searches the command history backwards for the most recent command containing the typed text; pressing Enter executed `pwd` again.

2.1.3 Built-in or External

```
$ type cd ; type ls ; type echo ; which ls
cd is a shell builtin
ls is aliased to `ls --color=auto'
echo is a shell builtin
/usr/bin/ls

$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/
   local/games:/snap/bin:/snap/bin

$ echo $PATH | tr ':' '\n' | wc -l
10

$ nosuchprog ; echo "exit code: $?"
nosuchprog: command not found
exit code: 127
```

2.1.4 Finding Help

`ls --help | head` and `man -k "list directory"` worked. The `man -k` output lists `rust-ls`, `rust-dir` and `rust-vdir`, which shows that this Ubuntu version uses the Rust reimplementation of the core utilities (uutils coreutils); this is why the `ls --help` text looks different from the classic GNU version. `help cd` printed the documentation of the built-in `cd`. In `man ls`, searching `/sort` led to `--sort=<field>`, where `time (-t)` is listed.

2.1.5 Observations

1. `cd` and `echo` are built-ins; `ls` is an external program (`/usr/bin/ls`, used through the alias `ls --color=auto`). `cd` **must** be a built-in because the current working directory is a property of each process. An external program runs in a child process: it could change only its own directory and then exit, leaving the shell's directory unchanged.

2. `PATH` has 10 entries, but `/snap/bin` appears twice, so there are 9 distinct directories. `ls` is found in `/usr/bin`.

3. A command that is not found returns exit code **127**.

4. The option that sorts by modification time is `ls -t` (equivalent to `ls --sort=time`), newest first.

2.2 Part 2: What the Shell Does to the Line

The test files were created in `~/os-lab3/play`:

```
$ touch a.c b.c file1.txt file2.txt notes.txt "alfa beta.txt"
$ ls
 a.c              b.c         file2.txt   log2.txt   notes.txt
'alfa beta.txt'   file1.txt   log1.txt    log3.txt
```

The predictions below were written in `predictii.md` before running the commands.

2.2.1 Predictions and Real Outputs

Line 1.

```
$ echo *.c
a.c b.c
```

Prediction: Displays the names of all `.c` files.
Correct. The shell expanded `*.c` before `echo` ran.

Line 2.

```
$ echo file?.txt
file1.txt file2.txt
```

Prediction: Displays the filename matching this pattern.
Correct, but there are two matches, not one: `?` matches exactly one character, and both `file1.txt` and `file2.txt` fit.

Line 3.

```
$ echo [a-b]*
a.c alfa beta.txt b.c
```

Prediction: Displays filenames starting with `a` or `b`.
Correct, but I had not thought about `alfa beta.txt`, which also starts with `a`. `echo` prints it without quotes, so it looks like two separate words.

Line 4.

```
$ echo *.xyz
*.xyz
```

Prediction: Displays the literal pattern `*.xyz`.
Correct. When no file matches, bash leaves the pattern unchanged.

Line 5.

```
$ ls alfa beta.txt
ls: cannot access 'alfa': No such file or directory
ls: cannot access 'beta.txt': No such file or directory
```

Prediction: Lists existing paths and reports errors for missing ones.
Correct. Without quotes, the space splits the name into two arguments, and neither `alfa` nor `beta.txt` exists, so both produce errors.

Line 6.

```
$ ls "alfa beta.txt"
'alfa beta.txt'
$ ls alfa\ beta.txt
'alfa beta.txt'
```

Prediction: Lists the file named `alfa beta.txt`.
Correct. The quotes (or the backslash) keep the name as one argument.

Line 7.

```
$ echo '$HOME' "$HOME" \$HOME
$HOME /home/vboxuser $HOME
```

Prediction: Displays `$HOME`, the home directory path, and `$HOME`.
Correct.

Line 8.

```
$ echo {1..5} ; echo $((7 * 6))
1 2 3 4 5
42
```

Prediction: Prints `1 2 3 4 5`, then `42`.
Correct: brace expansion and arithmetic expansion.

Line 9.

```
$ echo "today is $(date +%A), I am $(whoami)"
today is Sunday, I am vboxuser
```

Prediction: Displays the weekday and current username.
Correct. Command substitution also works inside double quotes.

Line 10.

```
$ touch log{1..3}.txt && ls log*
log1.txt log2.txt log3.txt
```

Prediction: Creates and lists `log1.txt`, `log2.txt` and `log3.txt`.
Correct.

2.2.2 Observations

1. The predictions and outputs are compared above.
2. `echo` knows nothing about files; the names were produced by the **shell**. Before running a command, bash performs pathname expansion (globbing): it replaces `*.c` with the list of matching file names and passes that list to `echo` as arguments.
3. `ls alfa beta.txt` failed because the shell splits words on spaces, so `ls` received two arguments, `alfa` and `beta.txt`, and neither exists. Two fixes: quoting (`ls "alfa beta.txt"` or `ls 'alfa beta.txt'`) or escaping the space (`ls alfa\ beta.txt`). Tab completion inserts the backslash automatically.
4. **Single quotes** keep everything literal: no variable, command or arithmetic expansion (`'$HOME'` gives `$HOME`). **Double quotes** prevent word splitting and globbing but still allow `$VAR`, `$(...)` and `$((...))` (`"$HOME"` gives `/home/vboxuser`).

2.3 Part 3: Redirection, Chaining and Jobs

```
$ ls /etc /nope > out.txt 2> err.txt
$ wc -l out.txt ; cat err.txt
237 out.txt
ls: cannot access '/nope': No such file or directory

$ ls /etc /nope > all.txt 2>&1 ; tail -2 all.txt
xml
zsh_command_not_found

$ ls /etc /nope 2>&1 > all2.txt ; tail -2 all2.txt
ls: cannot access '/nope': No such file or directory   <- on the terminal
xml
zsh_command_not_found

$ head -2 all.txt
ls: cannot access '/nope': No such file or directory
/etc:

$ grep nope all.txt all2.txt
all.txt:ls: cannot access '/nope': No such file or directory

$ echo first > notes.md ; echo second >> notes.md ; cat notes.md
first
second

$ wc -l < access.log
500
$ wc -l access.log
500 access.log

$ ls /nope 2> /dev/null || echo "failed with code $?"
failed with code 2
```

```
$ mkdir -p tmp && cd tmp && echo "I am in $(pwd)" ; cd ~/os-lab3
I am in /home/vboxuser/os-lab3/tmp

$ sleep 100 &
[1] 9276
$ jobs ; kill %1 ; jobs
[1]+  Running                 sleep 100 &
[1]+  Running                 sleep 100 &
[1]+  Terminated              sleep 100
```

2.3.1 Observations

1. `err.txt` received the error message (`cannot access '/nope'`), and `out.txt` received the listing of `/etc` (237 lines, including the `/etc:` header that `ls` prints when it receives several arguments).

2. `> all.txt 2>&1` vs `2>&1 > all.txt`.

   Prediction: the first redirects both outputs to the file; the second redirects errors to the terminal.

   This was correct. Bash processes redirections from left to right, and `2>&1` means "make standard error point to wherever standard output points *right now*".

   - `> all.txt 2>&1`: standard output is first sent to `all.txt`, then standard error is copied from it, so both go to the file. The error is at the *beginning* of `all.txt` (shown by `head -2`), which is why `tail -2` did not show it.
   - `2>&1 > all2.txt`: standard error is copied while standard output is still the terminal, and only afterwards standard output is moved to the file. The error appeared on the screen, and `grep` confirms that it is in `all.txt` but not in `all2.txt`.

3. `wc -l < access.log` prints only `500` because the shell opens the file and connects it to the standard input of `wc`; `wc` never receives a file name, so it has none to print. With `wc -l access.log`, the name is passed as an argument, `wc` opens the file itself and prints its name next to the count.

4. Chaining operators:

   - `;` runs the next command regardless of how the previous one ended;
   - `&&` runs the next command only if the previous one succeeded (exit code 0);
   - `||` runs the next command only if the previous one failed (non-zero exit code), as in `ls /nope ... || echo "failed with code $?"`, which printed code 2.

Jobs: `sleep 100 &` started a background job `[1]` with PID 9276. Right after `kill %1`, `jobs` still showed `Running`, because the signal had been sent but the process had not finished yet at that instant; bash then reported it as `Terminated`.

2.4 Part 4: Answering Questions with One-Liners

All commands are saved in `oneliners.sh`, one per question, with the question as a comment, and were run with `bash oneliners.sh` inside `~/os-lab3`.

2.4.1 Q1: Number of Failed Requests

```
grep -c FAIL access.log
```

```
100
```

The option `-c` counts the matching lines. **Answer: 100 requests failed.**

2.4.2 Q2: Different Requested Pages

```
cut -d' ' -f5 access.log | sort -u
```

```
/page0
/page1
/page2
/page3
```

The fifth field contains the requested page; `-u` removes duplicates after sorting. **Answer: 4 different pages**: `/page0`, `/page1`, `/page2`, `/page3`.

2.4.3 Q3: Number of Requests for Each Page

```
cut -d' ' -f5 access.log | sort | uniq -c
```

```
125 /page0
125 /page1
125 /page2
125 /page3
```

The pages are sorted first because `uniq` only counts adjacent equal lines. **Answer: 125 requests per page.** Check: $4 \times 125 = 500$, the total number of lines.

2.4.4 Q4: Users with the Most Failed Requests

```
grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn
```

```
15 user5
15 user3
14 user6
14 user4
14 user2
14 user1
14 user0
```

Answer: `user3` and `user5`, with 15 failed requests each. Check: $15+15+5\times14 = 100$, equal to Q1.

2.4.5 Q5: Requests Made by user3

```
echo "Total requests:"; grep -c 'user3' access.log; echo "Failed requests:";
    grep 'user3' access.log | grep -c FAIL
```

```
Total requests:
72
Failed requests:
15
```

Answer: user3 made 72 requests, of which 15 failed.

2.4.6 Q6: Last Three Failed Requests

```
grep FAIL access.log | tail -3 | cut -d' ' -f2,3
```

```
10:10 user0
10:15 user5
10:20 user3
```

The command keeps the last three failed lines and shows only the time and the user (fields 2 and 3).

2.4.7 Q7: Login Shells from /etc/passwd

```
cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn
```

```
40 /usr/sbin/nologin
 5 /bin/false
 2 /bin/bash
 1 /bin/sync
```

Answer: four shells appear. Most accounts are system accounts that cannot log in (`nologin` and `false`); only 2 accounts use `/bin/bash`.

2.4.8 Q8: Comparing the Number of Lines

Prediction: counts output lines in each listing; the counts may differ.

```
$ ls /etc | wc -l
236
$ ls -l /etc | wc -l
237
$ ls -l /etc | head -2
total 1152
drwxr-xr-x 5 root root 4096 Aug 26 05:30 ModemManager
```

Answer: when `ls` writes into a pipe instead of a terminal, it does not arrange names in columns: it prints one name per line in both cases, so both commands count the same 236 entries. The only difference is the first line of the long format, `total 1152` (the total number of disk blocks used by the listed files), which `ls -l` adds. This gives exactly one extra line.

2.5 Part 5: The Bash Report Script

The script receives the log file as an argument and displays the total number of requests, the number of failed requests, and the number of failed requests for each user.

```
#!/bin/bash
# usage: ./report.sh access.log

log=$1

if [ ! -f "$log" ]; then
    echo "no such file: $log" >&2
    exit 1
fi

total=$(wc -l < "$log")
fails=$(grep -c FAIL "$log")

echo "$total requests, $fails failed"

for user in user0 user1 user2 user3 user4 user5 user6
do
    count=$(grep "$user" "$log" | grep -c FAIL)
    echo "$user: $count failures"
done
```

2.5.1 Explanation of the Script

`log=$1` stores the first command-line argument. The `if` block checks with `-f` that the file exists; otherwise it prints an error on standard error and exits with code 1. The total is computed with `wc -l < "$log"`; input redirection is used so that the file name is not included in the output. The number of failed requests is computed with `grep -c FAIL`. Finally, the `for` loop goes through `user0` to `user6` and, for each one, counts the lines that contain both the user name and `FAIL`.

2.5.2 Running the Script

```
$ chmod +x report.sh
$ ./report.sh access.log
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures

$ ./report.sh nope.log ; echo "exit code: $?"
no such file: nope.log
exit code: 1
```

```
$ chmod -x report.sh ; ./report.sh access.log ; chmod +x report.sh
bash: ./report.sh: Permission denied

$ report.sh access.log
report.sh: command not found
```

The per-user failures match Q4 and add up to 100, equal to Q1.

2.5.3 Observations

1. `#!/bin/bash` (the *shebang*) tells the kernel which interpreter to use when the file is executed directly: the kernel runs `/bin/bash report.sh ...`. Without execute permission, `./report.sh` fails with `Permission denied`, as shown above by temporarily removing it with `chmod -x`. The script could still be run as `bash report.sh`, because then bash only reads the file.

2. `./report.sh` is required because the shell searches command names without a `/` only in the directories listed in `PATH`, and the current directory is not one of them. That is why `report.sh access.log` gave `command not found`. The `./` gives an explicit path, so no `PATH` search is performed. Keeping the current directory out of `PATH` is also a security measure: a malicious file named like a common command cannot be run by accident.

3. `>&2` sends the error message to standard error, keeping it separate from the normal output: if the output is redirected to a file or a pipe, the error still reaches the user and does not mix with the data. `exit 1` returns a non-zero exit code, so that the caller (`$?`, `&&`, `||`, other scripts, cron) knows that the script failed, as shown by `exit code: 1`.

3 CONCLUSIONS

During this laboratory work, bash was used both interactively and in a script. Tab completion and Ctrl+R history search make typing faster, while `type`, `which`, `PATH` and `man` explain where commands come from and how to use them; the difference between built-ins and external programs explains why `cd` must be part of the shell.

The most important idea of Part 2 is that the shell rewrites the command line before the program runs: globbing, quoting, variable, command and arithmetic expansion all happen in the shell, not in `echo` or `ls`. Writing predictions first made the differences visible, for example that `[a-b]*` also matches `alfa beta.txt`.

Part 3 showed that the order of redirections matters (`> file 2>&1` versus `2>&1 > file`) and how `;`, `&&` and `||` depend on exit codes.

In Part 4, small filters (`grep`, `cut`, `sort`, `uniq`, `wc`, `tail`) joined by pipes answered every question with one line, and the results were verified against each other (Q3 adds up to 500, Q4 adds up to Q1). It also showed the importance of sorting before `uniq` and corrected the initial explanation of Q8: the extra line comes only from the `total` line of `ls -l`.

Finally, `report.sh` turned the one-liners into a program that validates its input, reports errors on standard error and returns a meaningful exit code.